

BPL IEC validation

Author: Jan Peter Axelsson

Here I try to reproduce some results from Jonas Månssons master thesis report "Control of chromatography column in production scale", TFRT-5599.

The interaction with the model has changed slightly so that linear flow rate $LF = F/\text{area}$ is used. The column volume is now specified in terms of (cross-section) area times height. Thus by changing area the scale up works fine and what you do in practice. Further the time unit changed from seconds to minutes.

The model has 5 states as listed below, 3 states in liquid mobile phase and 2 states for the gel for bound proteins. In a few of Jonas figures also bound ions of the buffer are plotted but I do not do that. Bound ions of the buffer can be calculated from the difference of total binding capacity Q_{av} and bound protein and bound protein antagonist, see section 5.1 in the report.

The molecular weights listed below are not used in the simulations. They just give typical values and the molecular weight for bound protein and antagonist protein is just arbitrary here.

Parameters of the model in general as well as time scale are all arbitrary and focus is on qualitative aspects of the model.

The height of the column is 20 cm which is a common size in the industry.

```
In [1]: # Import the application model and context variables
        from BPL_IEC import *
```

Windows - run FMU pre-compiled JModelica 2.14

```
In [2]: # Import the general FMU_explore functions and adapt them to the application
        from fmu_explore_pyfmi import fmu_explore

        Adapted = fmu_explore(model, parValue, parLocation, parCheck, fmu_model, fmu_pr
                               MSL_usage, MSL_version, BPL_version, \
                               opts_std, simulationTime, timeDiscreteStates, stateValue, \
                               diagrams, ax, lines, external_function = profile)

        par = Adapted.par
        init = Adapted.init
        disp = Adapted.disp
        simu = Adapted.simu
        show = Adapted.show
        resetPen = Adapted.resetPen
        process_diagram = Adapted.process_diagram
        describe_general = Adapted.describe_general
        describe_parts = Adapted.describe_parts
        describe_MSL = Adapted.describe_MSL
        FMU_explore_info = Adapted.FMU_explore_info
        system_info = Adapted.system_info
```

```
profile = Adapted.external_function
SDG = Adapted.SDG
```

In [3]: `run -i BPL_IEC_explore.py`

Model for the process has been setup. Key commands:

- `par()` - change of parameters and initial values
- `init()` - change initial values only
- `simu()` - simulate and plot
- `newplot()` - make a new plot
- `show()` - show plot from previous simulation
- `disp()` - display parameters and initial values from the last simulation
- `describe()` - describe culture, broth, parameters, variables with values/units

Note that both `disp()` and `describe()` takes values from the last simulation and the command `process_diagram()` brings up the main configuration

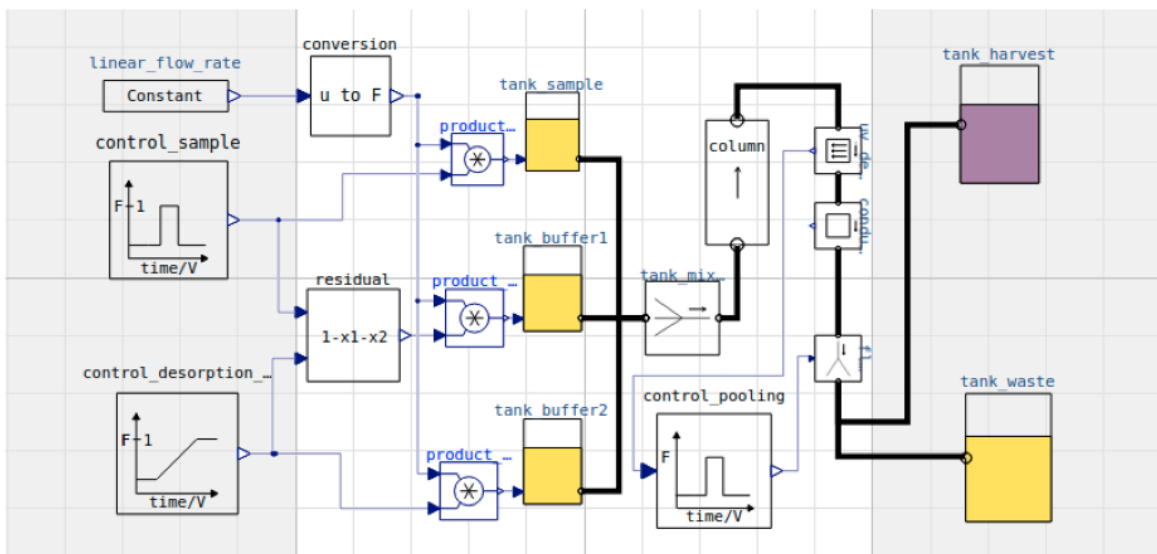
Brief information about a command by `help()`, eg `help(simu)`

Key system information is listed with the command `system_info()`

In [4]: `plt.rcParams['figure.figsize'] = [30/2.54, 24/2.54]`

In [5]: `# The process diagram is made outside Modelica to illustrate the configuration`
`process_diagram()`

No `processDiagram.png` file in the FMU, but try the file on disk.



In [6]: `#describe('chromatography'); print(); describe('Liquidphase')`

Loading or adsorption

The parameter notation and values are the same as in the referred report. However the flow rate is here denoted F while q in the report. The column is divided in $n=8$ sections and set at compilation time. The values are arbitrarily chosen in the report and the focus is on qualitative aspects of the model.

The simplified model describe only the column in terms of volume and does not distinguish a high column with a small diameter from a lower with larger diameter.

The parameters k_1 , k_2 , k_3 , k_4 and Q_{av} are given relative volume and with increased column volume a larger capacity is thus obtained.

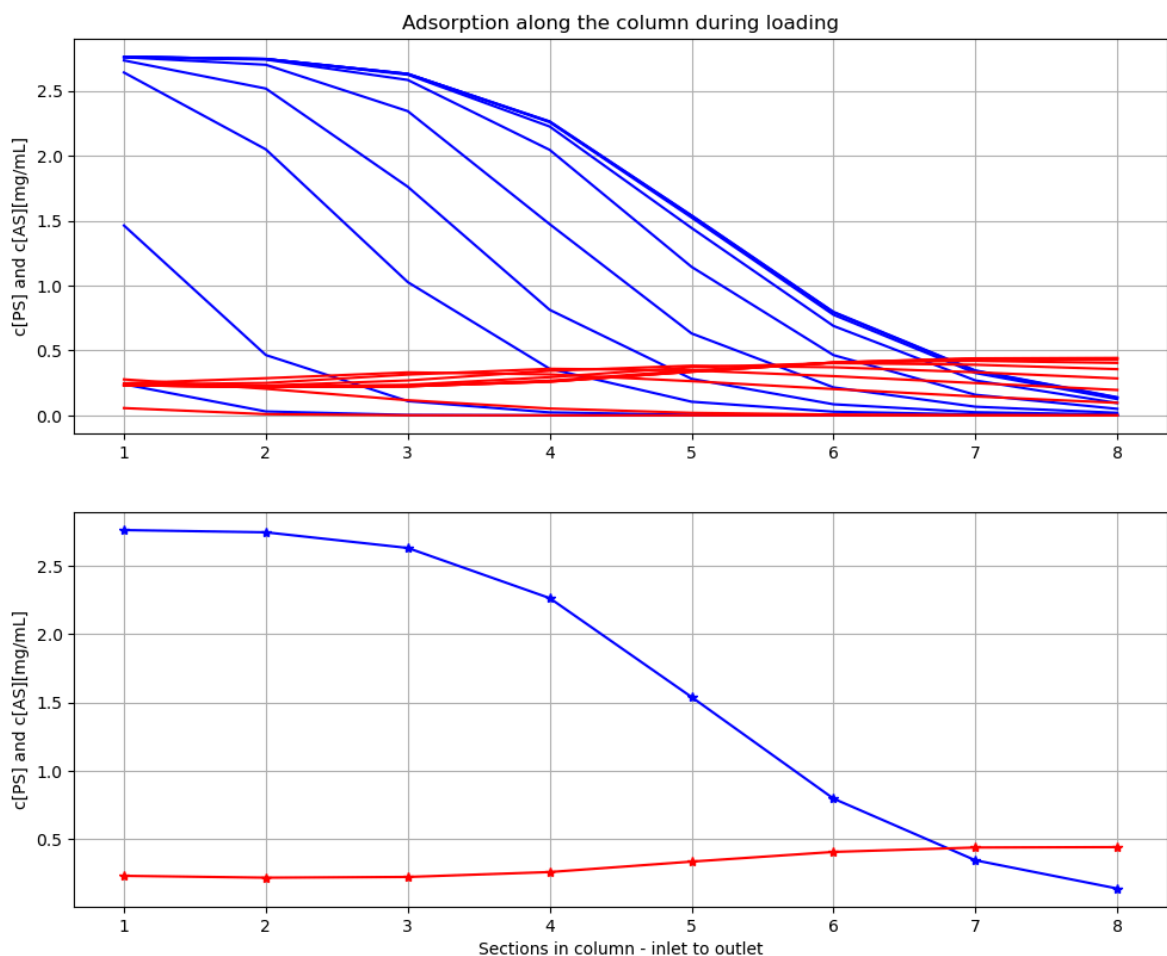
```
In [7]: # Loading of the column - try to reproduce Jonas figure 13.
newplot(title='Adsorption along the column during loading', plotType='Loading')

# Sample
par(P_in=1.0, A_in=1.0, E_in=0)

# Column properties
par(k1=0.3, k2=0.05, k3=0.05, k4=0.3, Q_av=3.0)
par(height=20, diameter=0.714)
par(x_m=0.3)

# Operation
par(E_in_desorption_buffer=8)
par(LFR=12)
#par(scale_volume=False)
par(start_adsorption=0, stop_adsorption=50)
par(start_desorption=150, stationary_desorption=450)
par(start_pooling=220, stop_pooling=450)

# Simulation
sim_res = simu(100)
```



The results are the same as Figure 13 in [1].

```
In [8]: # We just check that we had the same volume flow rate as Jonas
describe('F')
```

: 0.08

```
In [9]: describe('V')
```

Column volume total - derived : 8.008 [mL]

```
In [10]: model.get('column.x_m')
```

```
Out[10]: array([0.3])
```

```
In [11]: model.get('column.V_m')
```

```
Out[11]: array([2.40235705])
```

```
In [12]: #describe('column.n')
```

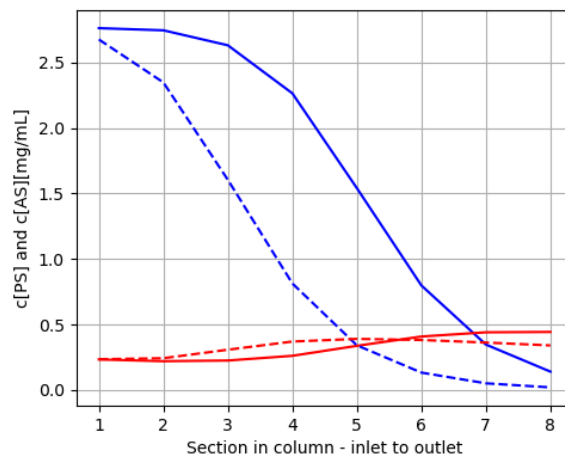
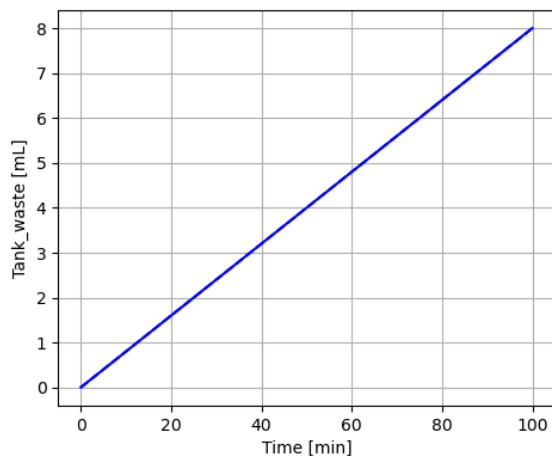
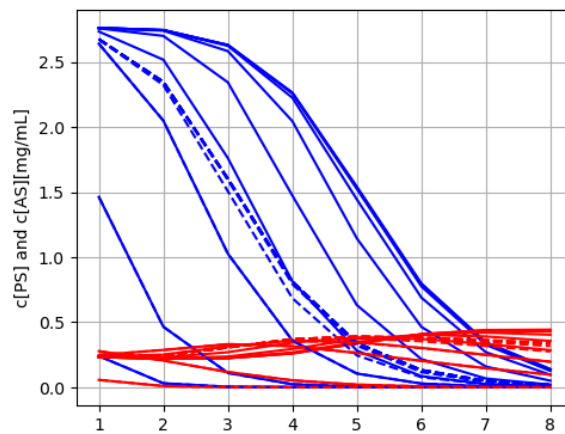
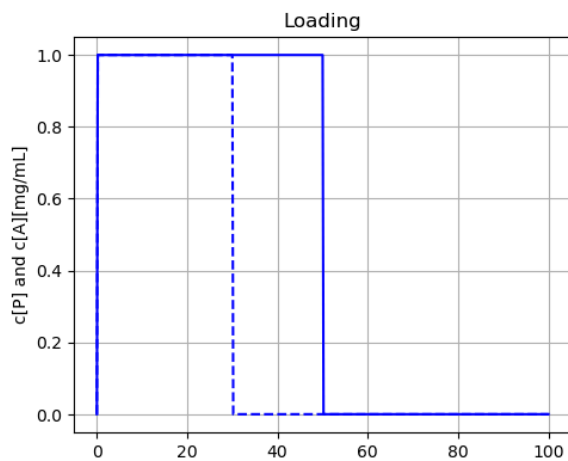
```
In [13]: model.get('column.column_section[1].V_m')
```

```
Out[13]: array([0.30029463])
```

```
In [14]: # Impact of shorter time for loading and then less material
newplot(title='Loading', plotType='Loading-combined')
show()

# Simulation with changed parameter t2
par(stop_adsorption=30); simu(100)

# Reset changed parameter
par(stop_adsorption=50)
```



To the left the inlet loading over time. To the right upper concentration along the column at different times and in steady states finally To the right lower concentrations along the column in steady state.

We see that a shorter time and then less material makes less of the column capacity used.

Note that the flow through the column is constant despite change from sample to just buffer 1, and shown in how the volume of the waste tank increase with time.

Elution or desorption

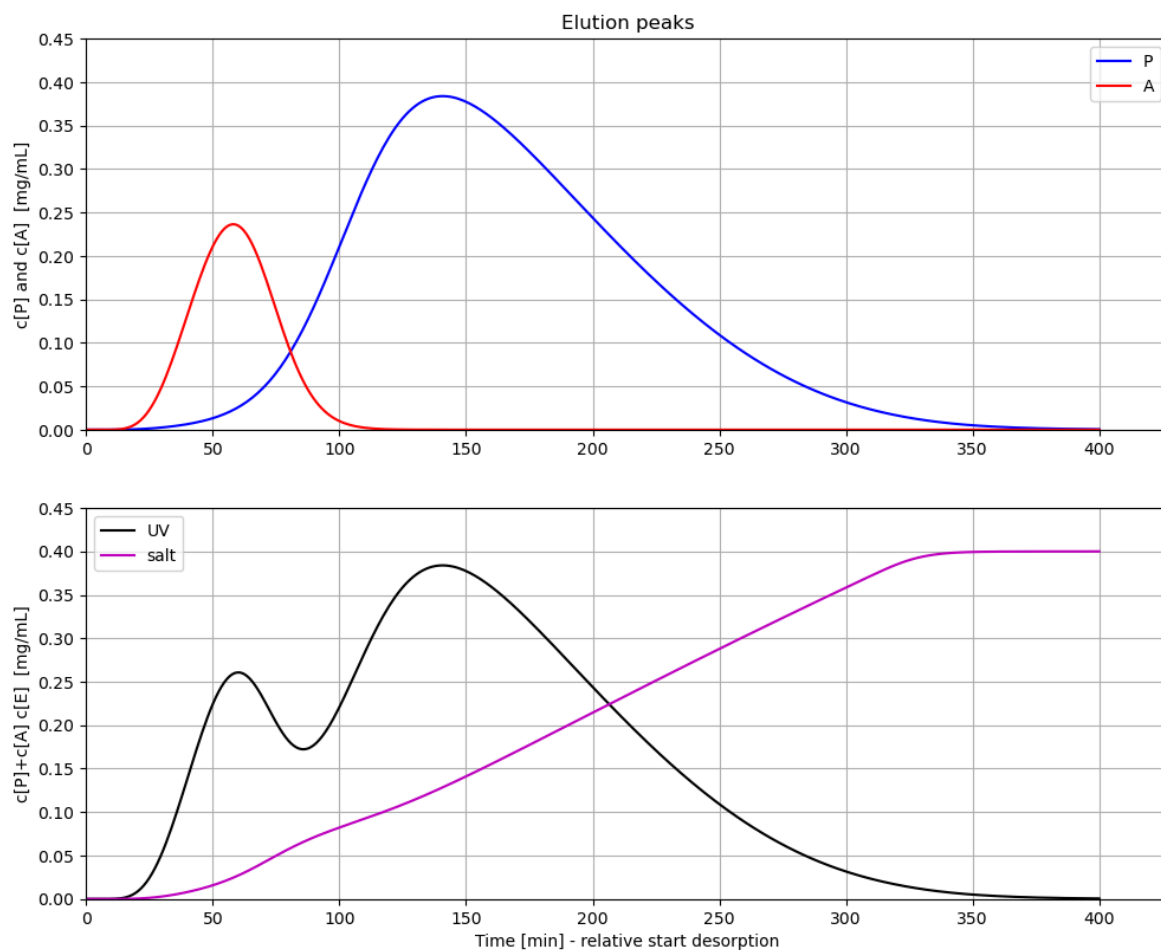
```
In [15]: # Elution of the column
newplot(title='Elution peaks', plotType='Elution')

# Sample
par(P_in=1, A_in=1.0, E_in=0)

# Operation
par(E_in_desorption_buffer=8)
par(LFR=12.0, start_adsorption=0, stop_adsorption=50, start_desorption=150, stat

# Simulation
simu(550)
```

Out[15]: <pyfmi.fmi_algorithm_drivers.FMIResult at 0x2114cee0170>



The results are the same as Figure 14 in [1].

The upper diagrams show the column outlet concentrations of P and A over time.

The lower diagram shows the sum (or possibly the UV signal) at column outlet as well as the salt concentration. We have some separation between the two peaks.

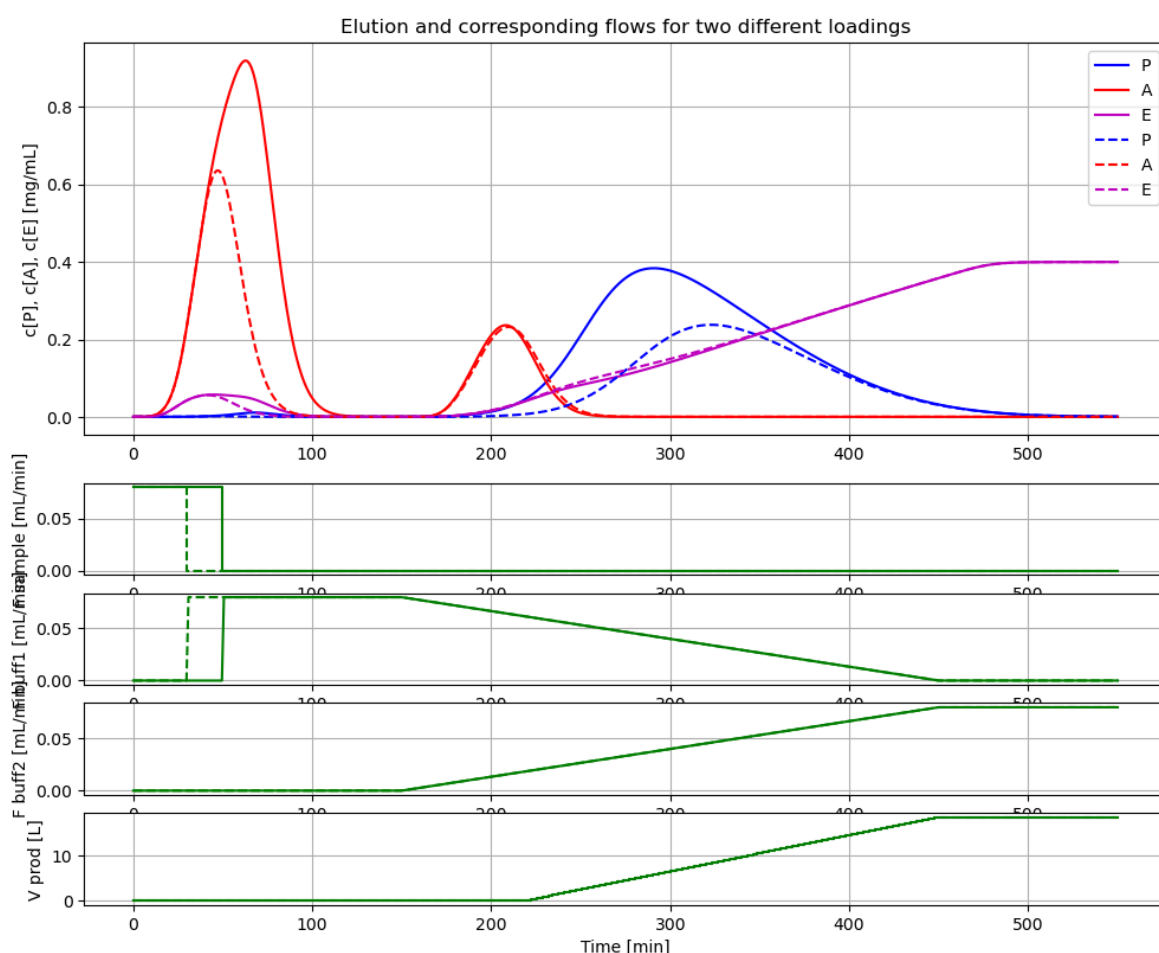
Note that the salt concentration deviates slightly from the linear increase between time 50 to 100. This is due to ion interaction with P and A in the column. This phenomenon can also be seen in real data. The ion-salt concentration is scaled with factor 0.05 to get comparable concentrations to P and A.

I have here simulated time 150 of adsorption and then started elution. Here is time counted as zero at time of start of elutions. Not sure how long Jonas simulated to get steady state before he did elution.

```
In [16]: # More complete visualization of the elution phase and the different flows
newplot(title='Elution and corresponding flows for two different loadings', plot
par(stop_adsorption=50); simu(550)

# Simulation with changed parameter t2
par(stop_adsorption=30); simu(550)

# Reset changed parameter
par(stop_adsorption=50)
```



Here a diagram that shows the peaks at the outlet as shown in the previous diagram. Below the flow rates of the three different sources. Here time is 0 at start of adsorption and elution starts at time 150.

Automatic pooling based on UV-measurement is tested in another notebook.

Change of resin properties

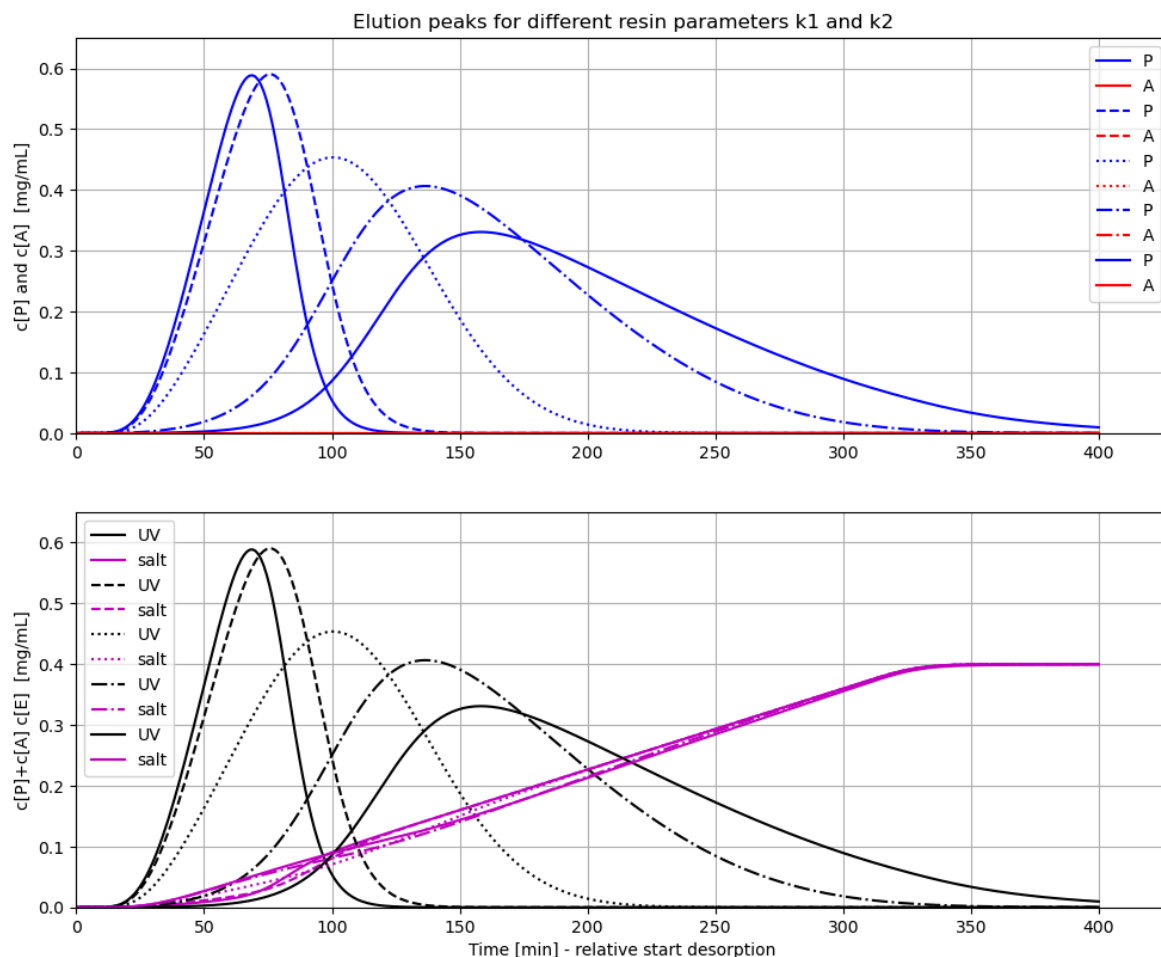
```
In [17]: # Elution of the column
newplot(title='Elution peaks for different resin parameters k1 and k2', plotType

# Sample
par(P_in=1, A_in=0.0, E_in=0)

# Operation
par(E_in_desorption_buffer=8)
par(LFR=12.0, start_adsorption=0, stop_adsorption=50, start_desorption=150, stat

# Simulations
par(k1=0.05, k2=0.50); simu(550)
par(k1=0.05, k2=0.25); simu(550)
par(k1=0.05, k2=0.05); simu(550)
par(k1=0.25, k2=0.05); simu(550)
par(k1=0.50, k2=0.05); simu(550)

# Adjust diagrams
ax[0].set_ylim([0, 0.65])
ax[1].set_ylim([0, 0.65])
plt.show()
```



The results are the same as Figure 17 in [1].

Summary

Three important diagrams Figure 13, 14 and 17 in the original report [1] were reproduced and the implementation in Modelica used here is considered validated.

The model is now extended with an improved parametrization of the column that matches industrial practice.

Acknowledgement

The author thanks Karl Johan Brink for sharing his know-how of chromatography operation. He has especially given input to how to parametrize the model in terms often used in industrial practice.

References

1. Månsson, Jonas, "Control of chromatography column in production scale", Master thesis TFRT-5599, Department of Automatic Control, LTH, Lund Sweden, 1998.
2. Pharmacia LKB Biotechnology. "Ion Exchange chromatography. Principles and Methods.", 3rd edition, 1991.

Appendix

In [18]: `describe('MSL')`

MSL: RealInput, RealOutput, Constants, Hysteresis, CombiTimeTable, Types

In [19]: `system_info()`

System information

- OS: Windows
- Python: 3.12.11
- Scipy: not installed in the notebook
- PyFMI: 2.21.0
- FMU by: JModelica.org
- FMI: 2.0
- Type: FMUModelCS2
- Name: Column_system
- Generated: 2025-07-25T19:11:26
- MSL: 3.2.2 build 3
- Description: Bioprocess Library version 2.3.1
- Interaction: FMU_explore version 1.1.6

In []: